SMAD: Robust DGA detection approach based on Improved Attention Mechanism

Shaoxuan Yun[a], Yuchao Zhang[a,*], Qianrui Yuan[a]

[a]School of Computer Science (National Pilot Software Engineering School), Beijing University of Posts and Telecommunications, Beijing, China

Abstract

Domain Generation Algorithms (DGAs) are techniques used by malware to algorithmically generate large numbers of domain names for establishing command-and-control (C&C) communication. By continuously querying dynamically generated domains, malware can evade blacklist-based defenses and maintain resilient C&C channels posing persistent threats to DNS security. Although recent Deep Neural Networks (DNN)-based detectors have achieved promising performance in offline evaluations, their effectiveness in real-world DNS environments is limited by two key challenges. First, detection models are vulnerable to adversarially crafted DGA domains that deliberately manipulate character patterns to evade detection models. Second, models often generalize poorly to previously unseen DGA families whose generation rules differ significantly from those observed during training. To address these challenges, we propose SMAD, a DGA detection approach designed to improve robustness against adversarially crafted domains and generalization to previously unseen DGA families. To counter adversarial DGAs that manipulate local character patterns, SMAD models character interactions beyond local dependencies, enabling the capture of meaningful structural features within a global context. To enhance generalization, SMAD focuses on learning stable, globally consistent character dependencies rather than family-specific heuristics, allowing the model to generalize effectively to unseen DGA variants. In addition,we implement a prototype of a real-time DGA detection system integrated into the DNS resolution path to support practical deployment. We evaluate SMAD on DGArchive, the largest publicly available DGA dataset, under regular detection, adversarial detection, and unseen-family settings. Experimental results show that SMAD achieves up to 96% detection accuracy in open-world scenarios and significantly outperforms state-of-the-art baselines in adversarial detection, attaining an AUC of 0.72. SMAD further demonstrates stable generalization performance under varying class imbalance ratios. Furthermore, system-level evaluation under real-world deployment conditions shows that the prototype sustains over 500 queries per second (QPS) with millisecond-level inference latency under continuous high-concurrency DNS workloads, demonstrating the practicality of SMAD for real-time DNS defense.

Keywords: Domain Generation Algorithm, Adversarial and Generalized DGA Detection, DNS-layer Security

1. Introduction

Domain Generation Algorithms (DGAs) enable malware to maintain reliable command-and-control (C&C) communication by circumventing network level defenses. [1, 2]. Instead of relying on fixed domain names that can be easily blacklisted, DGAs algorithmically generate large volumes of pseudo-random domain names over time, enabling malware to dynamically discover attacker-controlled servers through DNS resolution [3, 4]. As shown in Figure 1, in a typical DGA-based communication workflow, an infected host repeatedly issues DNS queries for algorithmically generated domains, only a small fraction of which are actually registered and activated by the attacker. The remaining queries result in failed resolutions, forming a characteristic pattern of high-volume, transient, and rapidly evolving DNS requests that is difficult to suppress using static filtering mechanisms.

Currently, DGA domain detection methods are primarily divided into rule-based engineering methods and Deep Neural Networks (DNN)-based methods. Rule-based methods (such as blacklists [5], random forests [6], clustering [7], etc.) rely

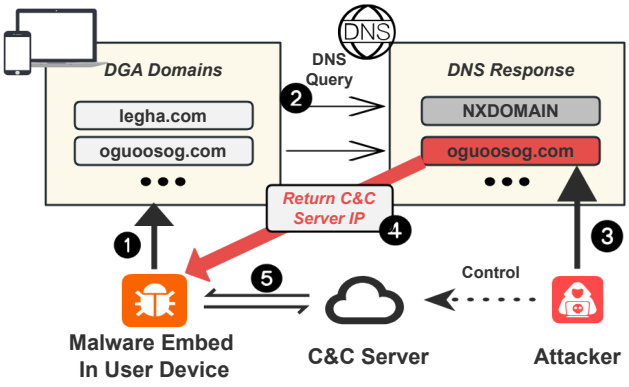


Figure 1: A typical communication workflow of DGA based malware is illustrated. ❶ The malware periodically generates a large number of pseudo-random domain candidates on the compromised host using a domain generation algorithm, aiming to evade conventional detection and blacklist-based filtering mechanisms. ❷ The host issues DNS queries for these generated domains to probe their availability, most of which are intentionally unregistered and result in failed resolutions. ❸ The attacker only needs to register a small subset of the generated domains and deploy a C&C infrastructure. ❹ When a queried domain is successfully resolved, the DNS response returns the IP address of an attacker-controlled C&C server, allowing the malware to identify a reachable target. ❺ Based on the resolved C&C server address, a covert and persistent communication channel can then be established between the compromised device and the attacker, enabling command execution, data exfiltration, and other malicious activities.

*Yuchao Zhang (yczhang@bupt.edu.cn) is the corresponding author.

on carefully designed features by experts and require constant updates to keep up with the rapid evolution of DGA technology [8]. These methods struggle to cope with the highly dynamic and unpredictable nature of DGA domains [9]. To address these limitations, DNN-based approaches have rapidly advanced, offering a more adaptive and automated solution. By leveraging large datasets, these methods can accurately and autonomously identify the characteristics of DGA domains without relying on manually defined rules or features [10]. Numerous studies [8, 11, 12] have demonstrated that DNN-based methods achieve state-of-the-art (SOTA) accuracy in DGA detection. [13, 14] were the first to introduce DNN methods into DGA detection, employing Long Short-Term Memory (LSTM) models. In addition, some studies [15, 16] have utilized Convolutional Neural Networks (CNN) and their variants as classifiers, employing character-level CNN to detect DGA domains and classifying them.

However, existing DNN-based DGA detection approaches still face several challenges, which are illustrated in Figure 2. Specifically: 1) Vulnerability to adversarial DGAs. Recent studies [17, 18, 19] have shown that attackers can generate adversarial DGA domains by applying adversarial perturbations to benign domain templates. As illustrated in the upper-left corner of Figure 2, these adversarial domains preserve strong structural similarity to legitimate domains while remaining malicious, making them difficult to distinguish from benign traffic. Most SOTA DNN-based detectors, such as CNNs and LSTMs, primarily focus on local character patterns and short-range dependencies, and thus fail to capture long-range contextual relationships. Consequently, their detection performance degrades significantly when facing adversarially crafted DGA domains. 2) Limited generalization across DGA families. As illustrated in the upper-left corner of Figure 2, different DGA families exhibit diverse generation rules and structural characteristics, including different character distributions and syntactic patterns. Due to the limited coverage of training data, detection models tend to overfit to known families and struggle to generalize to previously unseen DGA variants, resulting in degraded performance in open-world scenarios. These challenges motivate the design of a DGA detection model that can effectively capture global contextual information, establish robust character-level dependencies, and generalize across adversarial and unseen DGA domains.

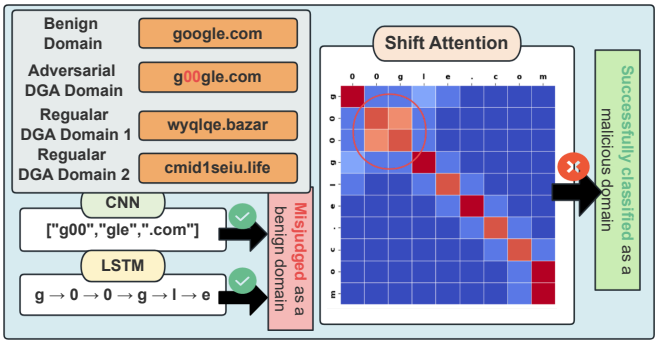


Figure 2: Examples of different types of DGA domains (adversarial DGA domains and two types of regular DGA domains) and the classification mechanisms of CNN, LSTM, and shift-attention.

To address the challenges, we propose SMAD, a DGA detection approach designed to improve robustness against adversarially crafted domains and generalization to previously unseen DGA families. First, to counter adversarial DGAs that mimic benign domain structures, SMAD explicitly models character interactions beyond local patterns. It introduces a Shift-Attention mechanism that reallocates attention from noisy or padded tokens to discriminative character relationships, enabling the model to capture meaningful local structures within a global context. This design reduces reliance on superficial character patterns that can be easily manipulated by adversarial obfuscation. Second, to mitigate overfitting to specific DGA families and enhance cross-family generalization, SMAD emphasizes globally consistent structural features rather than family-specific heuristics. By learning stable character-level dependencies shared across diverse generation rules, the model achieves more robust representations when encountering previously unseen DGA variants. In addition to its algorithmic design, SMAD is oriented toward deployment. Thus, we implement a real-time DGA detection prototype integrated into the DNS resolution path using dnsdist and Bind9, enabling malicious domain queries to be intercepted before command-and-control communication is established.

We evaluate SMAD from two complementary perspectives: detection effectiveness and deployment feasibility. For detection effectiveness, we conduct large-scale experiments on DGArchive to assess SMAD under regular DGA detection, adversarial DGA detection, and unseen-family generalization settings. Across these scenarios, SMAD consistently outperforms the state-of-the-art baselines, achieving up to 96% accuracy in regular detection, an AUC of 0.72 against adversarial DGAs, and an average accuracy of 93% in generalization experiments. To assess the feasibility of deployment, we further integrate SMAD into a real-world DNS detection prototype within a laboratory network environment and evaluate its system-level performance. The prototype operates inline on the DNS resolution path. It sustains a throughput of over 500 queries per second (QPS) with millisecond-level inference latency, demonstrating that SMAD can be practically deployed for real-time DGA defense in operational DNS environments.

To sum up, the main contributions of this paper can be summarized as follows:

- We systematically analyze the failure modes of existing DGA detection models under adversarial manipulation and open-world deployment, revealing that high offline detection accuracy alone is insufficient to ensure robustness against adversarial DGAs or reliable generalization to previously unseen families in real DNS environments.

- We propose SMAD, a DGA detection approach that improves robustness and generalization by explicitly modeling discriminative character dependencies while reducing the influence of non-informative patterns commonly exploited by adversarial domain generation. This design enables more stable detection under both adversarial perturbations and distribution shifts across DGA families.

- We demonstrate the effectiveness and practicality of SMAD through comprehensive detection performance evaluation and real-world deployment. Extensive experiments show that SMAD consistently outperforms SOTA methods in adversarial and open-world settings, while a DNS-layer prototype deployment confirms that SMAD can operate in-line detection with high throughput and millisecond level latency.

2. Background

In this section, we provide background knowledge about DGA domain name and threat model.

2.1. DGA

Domain generation algorithms (DGAs) are widely used by malware to establish resilient communication channels with attacker controlled command-and-control (C&C) servers. A DGA deterministically generates a large number of candidate domain names based on predefined seeds, such as timestamps, pseudorandom values, or wordlists, thereby giving rise to distinct DGA families with characteristic generation rules [20]. Hosts infected with malware repeatedly issue DNS queries for these generated domains, while the attacker needs to register only a small subset of them to successfully establish a C&C connection. From the defender's perspective, this algorithm-driven domain generation mechanism causes malware behavior at the DNS layer to appear highly complex and difficult to predict. In other words, newly generated DGA domains continuously replace previously blocked ones, enabling malware to evade static defense mechanisms such as blacklists and signature-based filtering. More importantly, in real-world attack scenarios, a malicious domain often requires only a single successful resolution to establish a C&C communication channel [3]. This property mandates that DGA detection systems make security decisions before domain resolution, without relying on retrospective traffic analysis or historical context [21]. Under these constraints, a practical detection system must operate in real time, introduce minimal additional latency, and generalize effectively to continuously evolving domain generation patterns. These requirements fundamentally distinguish online DNS-layer DGA detection from offline analysis or flow-based approaches, and they impose stringent constraints on the complexity, robustness, and deployability of detection mechanisms.

2.2. Adversarial Examples in DGA

Adversarial examples refer to inputs that are intentionally perturbed with subtle yet carefully crafted modifications to mislead machine learning models into making incorrect predictions. In the context of DGA detection, this concept has been adopted to design adversarial domain generation strategies capable of evading existing detection models. Prior studies [17, 18, 19] have shown that attackers can use legitimate domain names as templates and apply fine-grained perturbations to their local structures such as character insertion, substitution, deletion, or permutation to generate adversarial DGA domains that closely resemble benign domains in both appearance and statistical properties (as illustrated in Figure 2). These approaches typically combine techniques such as generative adversarial networks (GANs) [17, 19] or reinforcement learning [18], enabling the generated domains to remain resolvable while substantially degrading the detection performance. From the perspective of detection models, the threat posed by adversarial DGAs lies not only in their adversarial construction, but more fundamentally in their ability to expose structural weaknesses in existing DGA detection approaches. Many DNN-based detection methods tend to rely heavily on local character patterns or short-range dependencies during training. When such local features are deliberately perturbed or camouflaged, the learned decision boundaries of the models can deteriorate significantly, leading to increased misclassification. Therefore, it is necessary to enhance DGA detection methods at the architectural level by strengthening their ability to model global dependency structures and reducing excessive reliance on local or surface-level character patterns, thereby improving robustness in complex and evolving environments.

2.3. Transformer and Attention Mechanism

In recent years, attention mechanisms have been widely adopted in sequence modeling due to their ability to capture dependencies between different positions under a global context, thereby alleviating the performance degradation of locality-based methods under adversarial perturbations and distribution shifts. As a result, attention provides a reasonable foundational framework for modeling the structural characteristics of DGA-generated domain names. However, in high-noise character sequences, standard attention mechanisms can still be distracted by random characters or padding positions, making it difficult to consistently distinguish discriminative structural patterns. Existing efficient attention variants (e.g., Linear Attention, Performer, and Sparse Attention) primarily focus on reducing computational complexity for long-sequence scenarios, whereas the core challenge in DGA domain detection lies in robustly capturing key structural patterns under noisy and weakly semantic conditions. This observation motivates us to explore more targeted regulation of attention distributions to enhance their discriminative stability in noisy environments.

3. Threat model

As illustrated in Figure.3, we consider a typical local area network (LAN) environment in which benign hosts coexist with hosts infected by malware. On an infected host, malicious software runs without the user's awareness and embeds a DGA to periodically generate a large number of candidate domain names. These domains are then queried through the local DNS infrastructure in an attempt to establish communication with attacker-controlled C&C servers. Once a domain is successfully resolved, the malware can retrieve remote commands and execute subsequent malicious actions, leading to information leakage, lateral movement, or full system compromise. In our threat model, malicious DNS queries and legitimate DNS

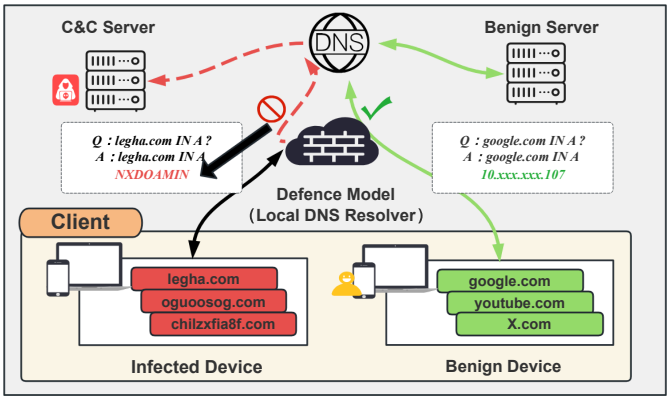


Figure 3: Threat model for real-world DGA detection in operational DNS infrastructure. Malicious and benign DNS queries coexist and traverse the same local DNS resolver. Infected hosts leverage domain generation algorithms (DGAs) to produce large volumes of transient domains in an attempt to reach attacker-controlled C&C servers, whereas benign hosts query legitimate services. The defender is positioned at the DNS resolver and must perform online detection and enforcement based solely on domain strings prior to resolution, without relying on traffic history or endpoint information.

queries share the same DNS resolution path and underlying network infrastructure. The defense system cannot a priori distinguish whether a query originates from an infected host. Consequently, defensive decisions must rely solely on the queried domain name itself, rather than on host level context or retrospective traffic analysis. This assumption more closely reflects real-world deployment scenarios and significantly increases the difficulty of detection.

To evade detection mechanisms, attackers may adopt various domain generation strategies: (1) generating highly random domain names based on dynamic factors such as time, random seeds, or dictionary combinations and (2) generating adversarial DGA domains using strategies specifically designed to exploit the weaknesses of detection models. Moreover, attackers may dynamically adapt their domain generation rules in response to defensive strategies, leading to the continuous evolution of malicious domains over time. In our system, the defense module is deployed at the entry point of the local DNS resolution path, serving as a unified scheduling and decision-making node for DNS queries. All DNS requests from clients are first processed by the defense model for real-time analysis. If a domain is classified as benign, the request is forwarded to the downstream recursive resolver to complete the normal resolution process and return the response. If a domain is identified as malicious, the resolution request is immediately blocked at this stage (e.g., by returning an NXDOMAIN response), thereby severing the communication between the malware and the C&C server before a connection is established. This defense operates in an inline manner, requires no modifications on the client side, and does not affect the availability of legitimate DNS services.

Furthermore, we focus on the open-world setting, in which attackers may employ arbitrary, previously unseen, or newly emerging DGA schemes. The objective of the detection system is not to identify specific DGA families, but to determine whether a given domain name is generated by a DGA. Under this setting, the detection model must exhibit strong generalization capability to handle malicious domains that do not appear in the training data and continuously evolve, thereby meeting the requirements of DNS-layer security protection in real-world network environments.

4. System design

In this section, we give a detailed introduction to the prototype framework of the designed detection system.

4.1. Overview

The proposed system consists of two main components: a DNS monitoring module and a DGA detection module. First, the monitoring module is positioned at the entry of the DNS resolution path, where it continuously receives domain queries and maintains a unified input channel for subsequent processing. Upon receiving a resolver request, the monitoring module forwards the queried domain to the DGA detection module, and the final DNS response is decided based on the classification result. In this manner, the module preserves normal DNS service functionality while enabling online identification of suspicious queries. Second, the DGA detection module adopts the proposed classification method, SMAD, to distinguish benign domains from those generated by DGAs. SMAD is built upon an encoder-only Transformer architecture. It first maps the domain name character sequence into vector representations through a padding embedding layer that integrates character-level embeddings with positional encodings. A Shift-Attention mechanism is subsequently applied to reorder attention weights and suppress low-ranking scores, allowing the model to focus on semantically critical character patterns. The processed features are then passed through dropout and normalization layers, followed by a multi-layer perceptron to produce the final classification output. The detailed design and implementation of these two modules are described in the following subsections.

4.2. DNS monitoring module

In our system, the DNS monitoring module is positioned at the DNS forwarding layer and serves as the unified ingress for domain name queries in the proposed system. It acts as an inline DNS forwarding/proxy and enforcement point deployed at the entry of the DNS resolution path. Its primary role is to integrate domain inspection into the DNS query handling workflow while preserving the standard resolution process for benign traffic.This in-line placement is critical for DGA-based threats, since a malware instance may only require a single successful resolution to obtain a reachable C&C endpoint; therefore, the security decision must be made before the query proceeds through normal resolution, rather than via retrospective analysis. When a client issues a DNS query, the monitoring module intercepts the request and forwards the queried domain name to the DGA detection module for analysis. In doing so, it bridges model inference and real DNS operations by translating the classification outcome into concrete DNS handling actions. Based on the returned classification result, the monitoring module determines the subsequent handling strategy. Queries

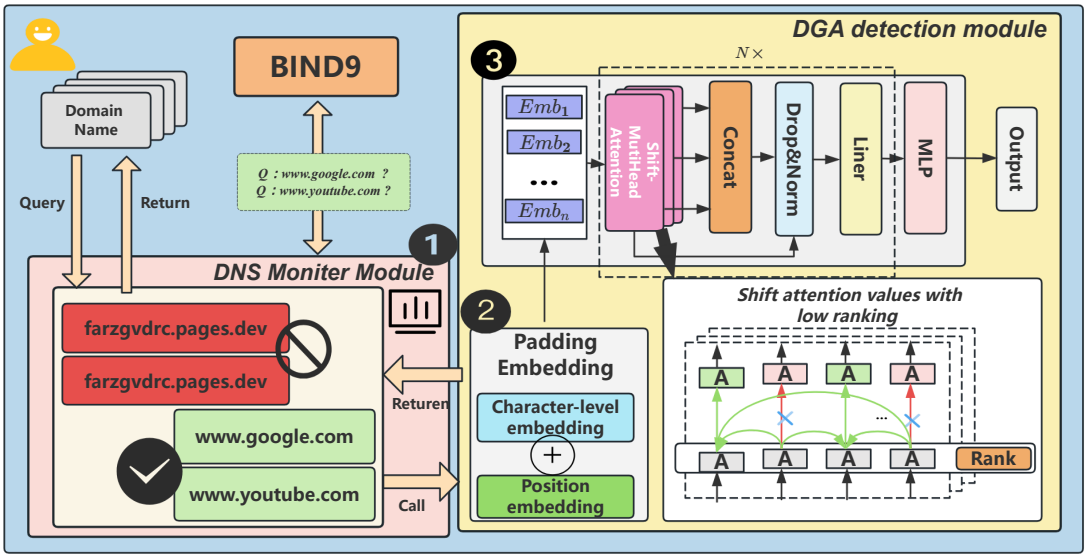


Figure 4: The figure shows the architecture of the system, which consists of two main modules: the Monitor Module ❶ and the DGA Detection Module ❷❸. The Monitor Module collects domain name queries and logs them using BIND9, updating the RPZ zone file. The DGA Detection Module processes the domain names through embedding techniques, while utilizing the Shift-Attention mechanism to improve feature extraction for DGA detection.

identified as benign are forwarded for normal name resolution, whereas queries classified as malicious are subject to predefined control actions, such as request blocking, response manipulation, or redirection to a controlled sinkhole. For example, malicious queries can be immediately blocked by returning NXDOMAIN or redirected to a sinkhole domain/IP under administrative control, thereby preventing C&C establishment at the DNS layer prior to any successful resolution. By embedding the detection decision into the query forwarding process, the monitoring module enables the system to prevent suspicious domain queries from progressing through the normal resolution pipeline. This design supports transparent deployment without client-side modifications, since the monitoring/enforcement logic is confined to the DNS forwarding layer and does not rely on endpoint instrumentation. This design allows malicious communication attempts based on DGA-generated domains to be mitigated at the DNS layer, without introducing additional dependencies on host-level context or retrospective traffic analysis. Meanwhile, the downstream DNS resolver remains unchanged for benign traffic, ensuring compatibility with DNS resolution and minimizing disruption to normal services.

4.3. DGA Detection Module

In this subsection, we give a detailed introduction to the designed detection approach (SMAD).

4.3.1. Padding Embedding

One-hot encoding, commonly used for string representation, transforms each word in a domain name into a vector with a length equal to the total number of characters, resulting in a two-dimensional vector for the entire domain name. However, this method leads to sparse model parameters and semantic ambiguity, impairing model training. Word2Vec, another prevalent technique, converts words into vectors based on contextual data from large corpora, capturing natural language semantics. Yet, it is unsuitable for DGA detection due to the chaotic nature of DGA domain names, which lack meaningful natural language patterns. N-gram encoding, which involves sliding window operations to generate byte fragments and count their frequencies, captures local text patterns but requires the entire training set for each prediction, hindering its effectiveness in DGA tasks. Consequently, we adopt character-level embedding, which aligns with RFC1035 specifications [22]. This method encodes each of the 40 permissible characters in domain names into unique numerical values, padding strings shorter than 255 characters with zeros, thus providing a more efficient and suitable approach for DGA detection. For each domain name, we use the formula (1) to represent:

$$\text{domain} = [x_1, x_2, \ldots, x_{255}] \tag{1}$$

After transforming the domain name into the vector described above, the vector is fed into an embedding layer. This layer maps the discrete numerical representation given in formula (1) into a continuous, dense embedding vector through a learnable matrix $E \in \mathbb{R}^{V \times D}$. Here, V denotes the number of embedding vectors (which in our model is a fixed constant size of 255), and D represents the dimensionality of the embedding vectors. During training, the embedding matrix E is continuously updated through its gradients, allowing the model to learn the subtle semantic differences between vocabularies and providing a richer feature representation for the subsequent detection model. Considering that the model employed in our domain name detection model lacks recurrence and convolution, we adopt the Position Encoding to capture and utilize positional information.

4.3.2. Domain identification classifier

The main objective of the DGA domain name classifier is to detect character pattern correlations within domain name sequences, thereby extracting valid features to distinguish benign from DGA domain names. As mentioned previously, there are

two issues with existing DGA domain detection. Firstly, recent studies [18, 17, 19] have shown that using adversarial example techniques, adversarial DGAs can generate domain names that closely resemble legitimate ones or subtly alter local features to deceive existing deep learning-based models, making it difficult to maintain high accuracy and evade detection mechanisms. Secondly, with the continuous emergence of new DGA families, detection models must be capable of identifying previously unseen domain name samples, thereby enhancing generalization ability. This necessity arises from the fact that, on the one hand, different types of DGAs exhibit distinct characteristics. For example, some generate domain names based on linguistic patterns, while others use random character sequences. Therefore, detection models need to accurately capture these characteristic differences to adapt to diverse DGA variants. On the other hand, not all DGA domains are present in the training set. Therefore, to prevent excessive reliance on training data, regularization strategies should be incorporated to mitigate the risk of overfitting. Consequently, by balancing feature adaptability and regularization, the model can maintain robust detection performance when confronted with unknown DGAs.

First, to better identify adversarial DGA domains, our work introduces an Attention mechanism to uncover hidden character patterns between tokens within domain name strings and capture dependencies among local features. Additionally, it enhances global contextual information, strengthening correlations between global features. This approach effectively mitigates the impact of localized modifications in adversarial DGA domain names, thereby improving detection robustness.

However, in the context of DGA domain detection, directly applying standard Attention poses two major issues: 1) The presence of random characters in DGA domains introduces strong noise, causing the model to attend to meaningless patterns instead of discriminative substrings. 2) Due to RFC-compliant padding requirements [22], the attention weights tend to disperse across padded positions, which degrades model performance.

To address these issues, we design a novel mechanism named **Shift-Attention**, which modifies the attention logits through a rank-dependent redistribution scheme. This mechanism selectively amplifies salient character interactions while suppressing noise tokens, effectively reshaping the attention distribution for robust DGA feature extraction.

We begin by computing the attention score matrix:

$$A = \frac{QK^T}{\sqrt{d}}, \tag{2}$$

where $Q, K, V \in \mathbb{R}^{n\times d}$ denote the query, key, and value matrices, and d is the hidden dimension. The i-th row A_i represents the attention logits assigned by the i-th query to all keys.

Next, instead of performing a uniform additive shift, which is invariant under softmax, we adopt a **rank-based shifting** strategy. Each row A_i is sorted in descending order, and the rank of each logit is defined as:

$$\text{rank}(A_{ij}) \in \{1, 2, \ldots, n\}, \tag{3}$$

where smaller rank values indicate more important interactions. To differentiate the contribution of tokens based on their importance, we assign each position a rank-dependent offset:

$$\Delta_{ij} = \alpha\left(1 - \frac{\text{rank}(A_{ij})}{n}\right), \tag{4}$$

where α is a scaling hyperparameter controlling the magnitude of adjustment. The shifted attention logits are then computed as:

$$\tilde{A}_{ij} = A_{ij} + \Delta_{ij}. \tag{5}$$

This design ensures that high-ranking logits (corresponding to key character interactions) receive positive reinforcement, while lower-ranking logits primarily associated with noise or padded positions are relatively suppressed. Because the offsets differ across ranks, the resulting attention distribution is no longer invariant under softmax translation, enabling effective redistribution of attention weights.

Finally, the Shift-Attention output is obtained by:

$$\text{Attention}_{\text{Shift-Attention}}(Q, K, V) = \text{softmax}(\tilde{A})V. \tag{6}$$

This rank-based adjustment enables the model to focus on discriminative DGA patterns while mitigating the impact of noisy or padded tokens, resulting in a more robust attention mechanism tailored to domain name characteristics.

Second, to address the issue of model generalization, we first employ multiple Shift-Attention layers, which utilize numerous independent attention heads to extract information from different subspaces. This enables the model to simultaneously focus on various features of DGA domain names, enhancing its ability to capture feature differences. For the i-th head, the computation is carried out using the following formula.

$$head_i = \text{Attention}^{\text{Shift-Attention}}(QW_i^Q, KW_i^K, VW_i^V) \tag{7}$$

The weight matrices W_i^Q, W_i^K, and W_i^V are all elements of $\mathbb{R}^{d\times d_h}$ represents the weight matrices for the i-th head, and D is the dimensionality of the output vector for each distinct head. If we denote the number of heads by hd, then the dimensionality of each head's output is calculated as $D' = \frac{D}{hd}$. After each head computes its result, these results are concatenated through a linear projection. We define $C(X)$ as the output after concatenating the results from each head. The definition of $C(X)$ is as follows:

$$C(X) = \text{Concat}(\text{head}_1, \ldots, \text{head}_{hd})W^O \tag{8}$$

Furthermore, we employ residual connections and Droppath techniques to mitigate gradient vanishing and overfitting in the model, thereby enhancing its generalization capability. Droppath involves randomly dropping some instances within the residual connections during model training, achieving differential model training, thereby reducing the risk of overfitting. Additionally, we employ layer normalization to normalize the activations of each layer, enhancing the stability of network training and accelerating convergence. SMAD contains 2 (where N equals 2) of the above blocks. Ultimately, we utilize a Multilayer Perceptron (MLP) layer that implements the standard

Softmax function as the output layer. The final output is denoted by $R(X)$. The computation formula for $R(X)$ is as follows.

$$R(X) = \text{MLP}(\text{LN}(X + \text{Dropout}(C(X)))) \quad (9)$$

5. Evaluation

In this section, we use benign and malignant domain names from the real world to evaluate the effectiveness of our proposed approach. We also compared the performance of our work with the advanced DGA detection algorithms.

5.1. Experimental Setup

Implementation. In our system implementation, dnsdist (v1.8.3 with LuaJIT 2.1) is adopted as the DNS monitoring and dispatching layer, while Bind9 (Berkeley Internet Name Domain version 9, v9.18.39) serves as the backend recursive resolver. Maintained by PowerDNS, dnsdist provides high-performance DNS load balancing and programmable query handling through Lua scripting, enabling in-line detection and decision-making before domain resolution. Our prototype is deployed on a server equipped with an Intel 11th Gen i5-11400 CPU, 16GB RAM, and a GeForce RTX 2080 GPU (8GB VRAM), which supports real-time inference and DNS request processing, providing a stable environment for subsequent evaluation. For model training and comparative experiments, we employ PyTorch as the deep learning framework. The training environment consists of an Intel Xeon Gold 5122 CPU @ 3.60GHz, 64GB RAM, and a GeForce RTX 4090 GPU (24GB VRAM), running Ubuntu 16.04 LTS with Anaconda 23.11.0 for development and execution. This configuration ensures sufficient computational capacity for model optimization and serves as the reference baseline for inference performance comparison in our system.

Dataset. In this study, we utilized both benign and malicious domain datasets to ensure comprehensive model evaluation. The benign dataset was derived from the Alexa Top 1 Million websites, representing legitimate domains ranked by global popularity. We assumed all samples from this dataset were non-malicious.

For malicious data, we constructed a diverse and challenging corpus comprising three subdatasets:

- Adversarial DGAs. We reproduced three state-of-the-art adversarial DGA generation methods (Khaos [17], CDGA [19], and PKDGA [18]), generating 30k adversarial domains in total (10k from each method).

- Regular DGAs. The standard DGA dataset was sourced from the 2016 DGA Archive [23], which includes 64 DGA families (e.g., Banjori, Bazar). We randomly selected 2 million samples, ensuring balanced representation across families. Notably, these 64 DGA families also include three high-quality dictionary-based DGA families, which closely mimic benign domains while still being malicious.

- Generalization Dataset. To evaluate generalization to unseen families, we employed the 2019 DGA Archive dataset, which extends the 2016 version with 25 newly added DGA families (totaling 89 families). The model was trained exclusively on the 2016 dataset and tested on the newly introduced families from 2019.

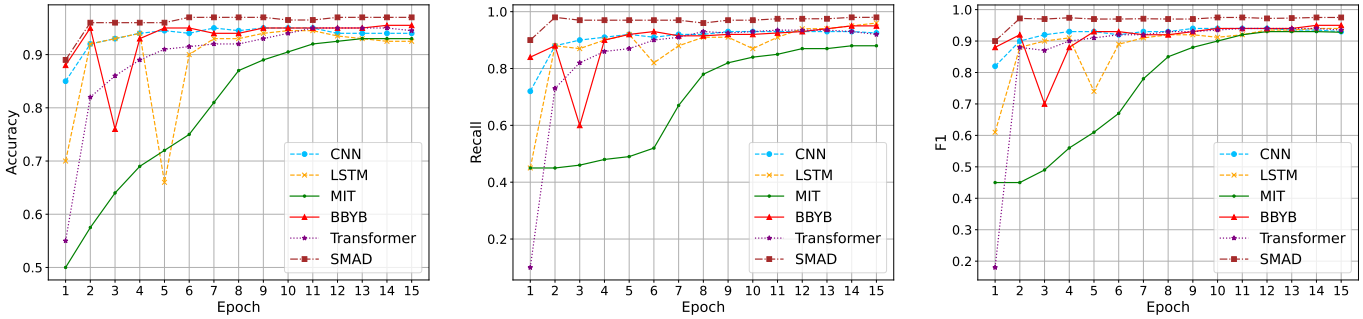
To systematically assess the model's robustness to class imbalance, we further constructed datasets with four benign-to-malicious ratios: 1:1, 1:3, 1:4, and 1:7. Besides, for each dataset configuration, we performed a 5-fold cross-validation procedure, ensuring that samples from the same DGA family did not appear in both training and testing splits to prevent data leakage. The training, validation, and test sets were divided in an 8:1:1 ratio. Model selection was based on the highest average validation F1-score across folds, which provides a statistically reliable performance estimate and mitigates overfitting.

Baseline. We replicated four DGA detection algorithms with advanced performance for comparison. In addition, we implemented the Self-Attention mechanism based on SMAD network structure as an ablation experiment with SMAD. The following is a brief introduction to these five algorithms:

- **Convolutional Neural Network (CNN)**[12]. The four parameters required by the CNN model are: the maximum index of the input size of the embedding layer, the maximum string length, the embedding dimension, and the output space dimension of the convolutional neural network. The overall structure of the CNN model is as follows: one embedded layer, five convolution layers of different sizes from 2*2 to 6*6, five global pooling layers, one layer of fully connected layers, two layers of Dropout layers, and the last layer of fully connected layers to output the final prediction results of the model.

- **Long ShortTerm Memory (LSTM)**[14]. The three parameters required by the LSTM model are: the maximum index of the input size of the embedding layer, the embedding dimension, and the hidden layer size that specifies the hidden state size of the LSTM unit. The overall structure of the LSTM model is as follows: one layer of embedding, one layer of LSTM processing input sequence data, one layer of Dropout, and the last layer of full connection is used to map the hidden state of LSTM output to the final prediction result.

- **MIT Hybrid Model (MIT)**[11]. The two parameters required by the MIT model are: the maximum index of the input size of the embedding layer, and the embedding dimension. The overall structure of the MIT model is as follows: an embedding layer, a one-dimensional convolution layer that performs convolution operations on embedded vectors, a pooling layer that performs pooling operations on the output of the convolution layer, a LSTM layer that processes input sequence data, and finally a fully connected layer that maps the hidden state of the output of MIT's LSTM layer to the final prediction result.



(a) Accuracy (b) Recall (c) F1

Figure 5: Experiment result of open-world scenario [positive data:negative data=1:1]

Table 1: Comparison of model performance in open world

	1:1			1:3			1:4			1:7		
	ACC	Recall	F1	ACC	Recall	F1	ACC	Recall	F1	ACC	Recall	F1
CNN	0.9538	0.9603	0.9532	0.9582	0.9726	0.9577	0.9581	0.9696	0.9575	0.9578	0.9686	0.9572
LSTM	0.9505	0.9496	0.9491	0.9575	0.9581	0.9564	0.9272	0.9540	0.9272	0.9510	0.9716	0.9508
MIT	0.9301	0.9620	0.9314	0.9622	0.9726	0.9615	0.9619	0.9745	0.9614	0.9500	0.9744	0.9499
BBYB	0.9433	0.9607	0.9431	0.9431	0.9645	0.9429	0.9550	0.9690	0.9544	0.9390	0.9445	0.9378
Transformer	0.9336	0.9422	0.9324	0.9274	0.8943	0.9228	0.9111	0.8717	0.9051	0.9223	0.9247	0.9206
SMAD	**0.9685**	**0.9624**	**0.9675**	**0.9663**	**0.9765**	**0.9658**	**0.9637**	**0.9773**	**0.9633**	**0.9610**	**0.9788**	**0.9607**

- **Bilbo hybrid model(BBYB)**[8]. Bilbo model is composed of the basic layer structure of ANN, CNN, and LSTM models. The three parameters required by Bilbo's model are: the maximum index of the input size of the embedding layer, the embedding dimension, and the convolutional neural network output space dimension. Bilbo model's overall structure is: CNN model layer structure, LSTM model layer structure. The output results of the two models, CNN and LSTM, form a mixed feature. The mixed feature is mapped to a lower-dimensional feature through the ANN model layer structure, and the lower-dimensional feature outputs the final prediction result through the last fully connected layer.

- **Transformer**. To better understand and validate the advantages of the Shift-Attention mechanism, we reproduced the original Self-Attention mechanism in order to conduct ablation experiments. We replaced the Shift-Attention mechanism in the SMAD network structure with the Self-Attention mechanism, ensuring that no other parts were altered. In this experiment, to adhere to the principle of controlling a single variable, we also set N = 2. Finally, we refer to this model as SA-Encoder for short.

To ensure fair comparison across all baselines, we performed grid-based hyperparameter tuning using the same search space. The learning rate was tuned in $\{1e^{-5}, 5e^{-5}, 1e^{-4}, 5e^{-4}\}$, batch sizes in $\{32, 64, 128\}$, and the embedding dimension in $\{64, 128, 256\}$. After tuning, we adopted a learning rate of $1e^{-5}$, a batch size of 64. In addition, for the rank-dependent offset in Eq. (4), we determine the scaling hyperparameter α via a lightweight validation-based search under the same 5-fold cross-validation protocol used throughout the paper. Specifically, we evaluate α over a small candidate set within the range of 10 to 40 and select the value that maximizes the average validation F1-score. For consistency and fair comparison, the selected α is fixed across all experiments. Unless otherwise stated, we use $\alpha = 25$ in all evaluations. For reproducibility, we additionally report the architectural hyperparameters of SMAD. The feed-forward network (MLP) uses a hidden size of 1024 with GELU activation and a dropout rate of 0.1. The model adopts 8 attention heads, and we further incorporate stochastic depth with a DropPath rate of 0.1 in both the attention and MLP residual branches.

Metrics. For the DGA domain name detection task, we use Accuracy, Recall, and F1 to measure model performance. These three metrics reflect the accuracy, completeness, and comprehensive performance of the classifier, respectively. In addition, in the adversarial DGA domain detection experiments, AUC is selected as the evaluation metric, as it is derived from the true positive rate (TPR) and false positive rate (FPR). The Receiver Operating Characteristic (ROC) curve is further employed to visualize the trade-off between TPR and FPR under different decision thresholds, providing an intuitive assessment of the model's discriminative capability. Together, AUC and the ROC curve effectively reflect the overall performance of the model in detecting DGA domains and are commonly used in research on adversarial domain generation algorithms [17, 18, 19].

5.2. *Regular DGA domain detection*

In this section, we evaluate the performance of the SMAD model using a dataset from the regular DGA domain. As described in Section 3, the detection system in an open-world ex-

periment focuses solely on determining whether a domain belongs to the DGA category. Therefore, we define the DGA detection task as a binary classification problem, categorizing all DGA domains into one class and all benign domains into another for model training and evaluation. For performance evaluation, in addition to selecting Accuracy as a metric, we further introduce Recall, as it measures the model's ability to detect DGA domains. In an open-world scenario, misclassifying a DGA domain as benign (false negative) could allow malware to successfully connect to its C&C server, posing severe security threats. Therefore, improving recall is critical for enhancing the security of the detection system. However, focusing solely on recall may lead to a decline in precision, increasing the false positive rate. To balance Accuracy and Recall, we introduce the F1 score as a comprehensive evaluation metric.

First, let us look at the performance of various models during the training process. Figure 5 shows the results of the three metrics on the test set as training epochs increase under a 1:1 data ratio. Initially, we observe the models' accuracy, which directly represents a model's capability in detecting DGA domains. Accuracy trends show that SMAD consistently improves, reaching nearly 1, significantly outperforming other models in DGA detection. Recall analysis indicates that while MIT and CNN models improve gradually, SMAD maintains an average of 0.95 despite fluctuations, demonstrating superior false-negative minimization. F1 score results further confirm SMAD's ability to balance accuracy and recall, consistently achieving the highest performance among all models.

After that, we also selected the best epoch from the training process in the open-world scenario for each model and used the validation set to observe the models' performance in relatively real-world settings. Table 1 displays the results of different models across various dataset ratios. As the balance between positive and negative data in the training set increasingly skews, the accuracy of all models declines. However, our designed SMAD model still maintains an accuracy of 0.96 in these conditions, further demonstrating SMAD's resilience to data imbalance, making it applicable for practical detection scenarios. As can be seen, LSTM and Transformer are clearly more suited to binary classification scenarios. Although their accuracy never reaches the level of SMAD, they can still maintain accuracies of around 0.95 and 0.92, respectively. Moreover, the BBYB model's accuracy is constrained by the training set's data ratio, performing poorly with only about 0.93 accuracy in the 1:7 dataset. Turning our attention to recall rates, we can see that all models perform relatively well, generally maintaining above 0.95. Still, SMAD continues to outperform existing algorithms across all data ratios, possessing the lowest misjudgment rate. Finally, through the F1 scores, it is evident that the SMAD model excels in balancing accuracy and recall, ensuring high accuracy while also maintaining a very low rate of misjudgments.

Although accuracy, recall, and F1-score provide useful insights into the performance of different detection models, these metrics only reflect classifier behavior at a fixed decision threshold. To enable a more comprehensive comparison, we further evaluate all models using Receiver Operating Characteristic (ROC) curves. Figure 6 illustrates the ROC curves of all models at their best-performing epoch under a 1:1 benign-to-DGA evaluation setting. The results show that SMAD consistently achieves higher true positive rates across almost all false positive rate regions, indicating a stronger overall discriminative capability. In contrast, MIT exhibits the weakest performance, with noticeably lower true positive rates at moderate and high false positive rates. CNN, LSTM, and BBYB achieve competitive performance, while Transformer demonstrates moderate performance between classical architectures and SMAD. These results indicate that SMAD maintains the most robust decision boundary among all baselines, further validating the effectiveness of the proposed mechanism in enhancing discriminability under realistic DNS conditions.

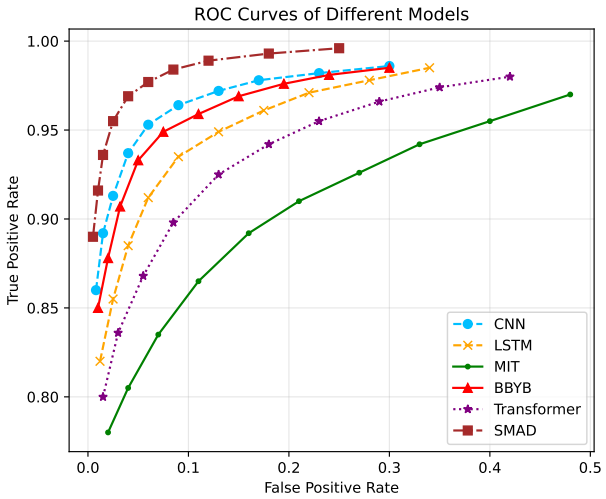
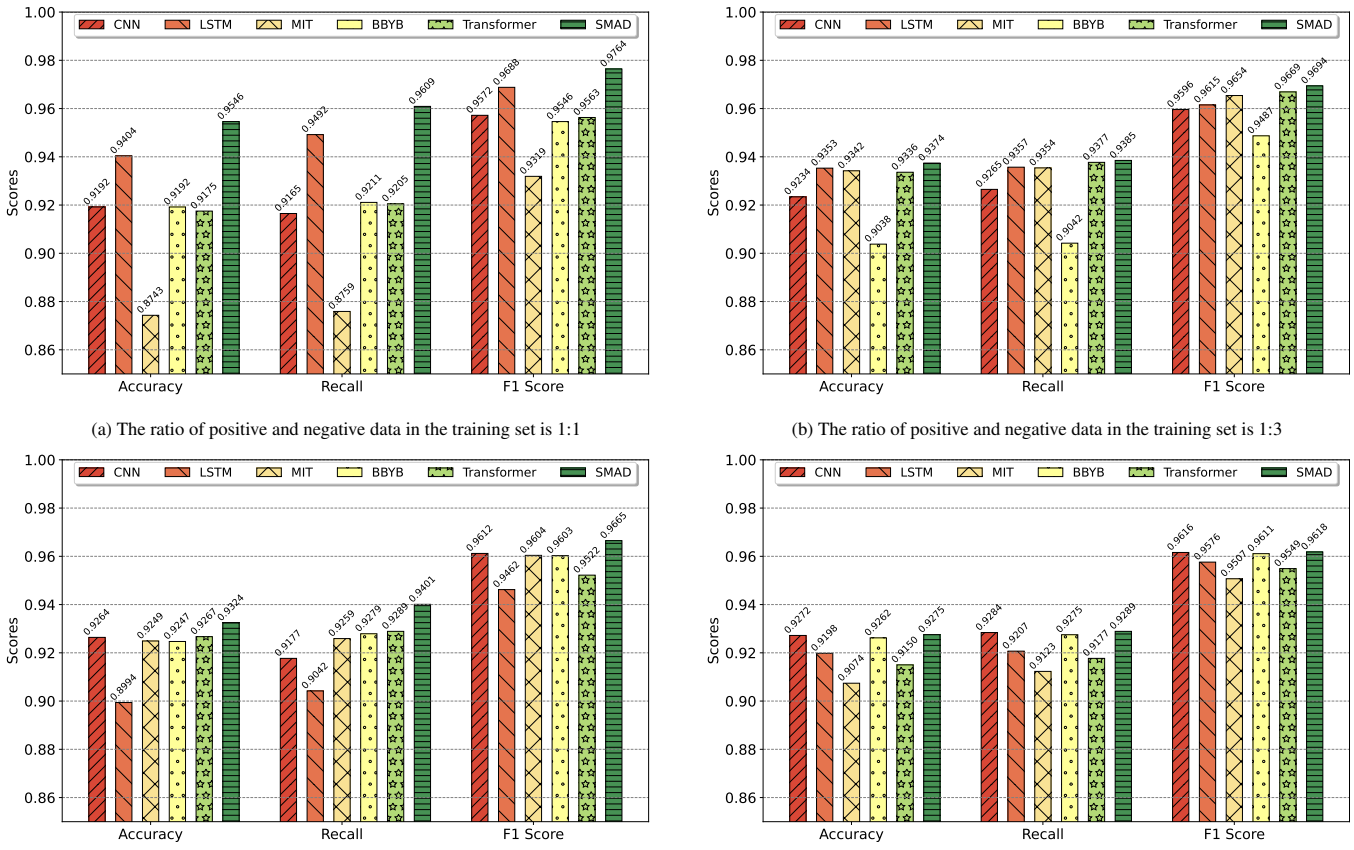


Figure 6: The ROC curve in open-world scenario.

5.3. *Adversarial DGA domain detection*

In this section, we evaluate the performance of different models for detecting adversarial DGA domains using adversarial DGA datasets. We adopted the models trained in Section 5.2 under the 1:1 positive-to-negative ratio setting as the detection models in this experiment, without adversarial training. Table 2 presents the AUC scores of different models across four types of adversarial DGA domain names, namely CDGA, Khaos, PKDGA, and DeepDGA. As mentioned above, the AUC score measures the model's ability to distinguish between DGA and non-DGA domains, with higher values indicating better performance. The SMAD and Transformer models, both incorporating self-attention mechanisms, consistently achieve the strongest performance across all datasets. Specifically, SMAD attains the highest AUC scores of 0.72, 0.67, 0.63, and 0.85 on CDGA, Khaos, PKDGA, and DeepDGA, respectively. The Transformer model follows closely, with AUC scores of 0.69, 0.61, 0.59, and 0.83 on the corresponding datasets. These results indicate that self-attention-based architectures are more robust in capturing discriminative patterns of adversarially generated DGA domains. The BBYB and MIT models show moderate performance. For example, they achieve AUC scores of 0.64 and 0.63 on CDGA, respectively, but their performance de-



(a) The ratio of positive and negative data in the training set is 1:1

(b) The ratio of positive and negative data in the training set is 1:3

(c) The ratio of positive and negative data in the training set is 1:4

(d) The ratio of positive and negative data in the training set is 1:7

Figure 7: Comparison of model generalization ability.

clines on Khaos and PKDGA. On the DeepDGA dataset, however, both models show some improvement, achieving AUC scores of 0.79 and 0.76, respectively, although they still remain below SMAD and Transformer. This suggests that DeepDGA-generated domains may preserve certain detectable patterns that can still be captured by these models, while the more challenging datasets such as Khaos and PKDGA remain harder to distinguish. By contrast, CNN and LSTM exhibit the weakest overall performance. Their AUC scores are 0.61 and 0.57 on CDGA, decreasing further to 0.53 and 0.47 on Khaos, and 0.49 and 0.50 on PKDGA, respectively. On DeepDGA, their performance improves to 0.78 and 0.63, but they still lag behind the best-performing methods. Overall, these results suggest that earlier baseline models are less effective in detecting adversarial DGA domains, whereas more advanced architectures with attention mechanisms, especially SMAD, demonstrate superior and more consistent robustness across different types of adversarial DGA attacks.

Table 2: AUCs Under Different Models (Higher is Better)

Model	CDGA[19]	Khaos[17]	PKDGA[18]	DeepDGA[24]
CNN	0.61	0.53	0.49	0.78
LSTM	0.57	0.47	0.50	0.63
MIT	0.63	0.46	0.44	0.76
BBYB	0.64	0.50	0.42	0.79
Transformer	0.69	0.61	0.59	0.83
SMAD	**0.72**	**0.67**	**0.63**	**0.85**

5.4. Model generalization ability experiment

In this section, we conduct experiments to evaluate the generalization ability of our model. As extensively demonstrated in Sections 5.2 and 5.3, SMAD performs well on both DGA datasets. However, as described in Section 3, detection models still face a significant challenge in practical scenarios: devices within distributed systems may become infected with malware using DGA families not present in the training set. This necessitates that the models possess adequate generalization capabilities to effectively handle such situations. We employed models trained in subsection 5.2, which were based on data from 2016. For our generalization experiments, we composed a prediction set using malicious domain names from 25 DGA families that appeared only in the 2019 dataset. Figure 7 shows the results of different models trained with varying dataset ratios. The experimental outcomes indicate that although all models experience a decline in performance when facing data imbalance, the SMAD model still exhibits the best performance, achieving an accuracy as high as 0.95 in a 1:1 dataset scenario. This implies that our designed model can effectively capture the character patterns in domain strings and utilize these features for DGA detection. Additionally, observing the F1 scores reveals that SMAD consistently maintains values above 0.96, further evidencing its excellent performance in DGA detection tasks. The result shows that SMAD is capable of being deployed in real-world applications, and even if devices are infected with the latest DGA families, SMAD can successfully detect them.

In summary, through experiments, we demonstrate that our

proposed model, SMAD, achieves optimal DGA detection results. SMAD not only excels in detecting conventional DGA domains but also maintains strong performance against highly obfuscated adversarial DGA domains. Furthermore, compared to other models, SMAD exhibits superior generalization capability when handling imbalanced datasets and previously unseen DGA domains.

5.5. *DGA family classification*

To further evaluate the capability of SMAD on the DGA family classification task beyond malicious/benign binary detection, and to maintain consistency with the binary-classification setting, we conducted comparative experiments under four positive-to-negative sample ratios, namely 1:1, 1:3, 1:4, and 1:7. For each model, we selected the checkpoint with the best performance during training and evaluated it on the test set to examine its best achievable performance. Table 3 presents the results of different models under various dataset ratios.

From the perspective of accuracy, SMAD consistently delivered superior performance across all dataset ratios. In particular, under the highly imbalanced 1:7 setting, its accuracy still remained at 0.9292, significantly outperforming the other baseline models. This further indicates that, even under imbalanced data distributions, SMAD retains high precision and reliability in domain classification tasks. In terms of recall, although all models exhibited a decline as the dataset became increasingly imbalanced, SMAD still maintained a recall of 0.7645, suggesting that it could still identify the majority of malicious domain names even under challenging detection conditions. Finally, SMAD also achieved strong F1 scores, demonstrating that it maintains a favorable balance between precision and recall.

It is also worth noting that the results in the table clearly show that the LSTM and BBYB models exhibit noticeable fluctuations in all three evaluation metrics as the data ratio changes. In particular, although the BBYB model showed a slight improvement when the dataset ratio changed from 1:1 to 1:4, its accuracy dropped to 0.8441 once the data became imbalanced, indicating that this model is strongly dependent on the class distribution of the training set and is therefore less suitable for the DGA family classification task. In addition, we observed that both the CNN and Transformer models performed unsatisfactorily in the multi-class setting, suggesting limited effectiveness for fine-grained DGA classification.

5.6. *Insight of Shift-Attention mechanism*

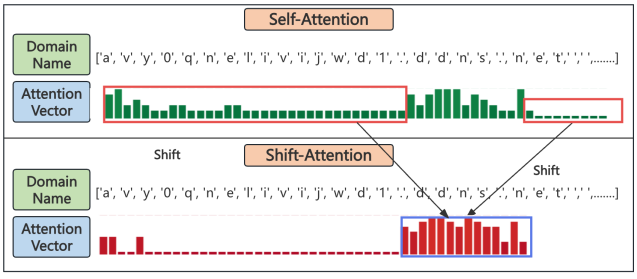


Figure 8: The difference between Self-Attention and Shift-Attention.

In this subsection, we further analyze the effectiveness of the Shift-Attention mechanism. Using a domain from the CoreBot family as an example, the Self-Attention model misclassified it as benign, while the Shift-Attention model correctly identified it as malicious. According to DGArchive[23], CoreBot domains often contain random characters followed by 'ddns.net'. We input this example into the SMAD network with both mechanisms and analyzed the attention vector values from the encoder's final layer (Figure 8).

Both mechanisms focus on the key feature 'ddns.net', but the Shift-Attention mechanism assigns it significantly higher attention values. This improvement stems from two factors: 1) padding introduced to comply with RFC standards, and 2) noise from random characters. While Self-Attention attempts to minimize padding-related attention, it cannot eliminate it entirely. In contrast, Shift-Attention redistributes lower-ranked attention weights to key features, reducing padding-induced noise. Additionally, Shift-Attention balances attention to random characters, ensuring they neither dominate nor disappear, thereby enhancing generalization. In this example, Self-Attention overemphasizes certain random characters and padding, leading to misclassification.

Overall, this real-world example demonstrates the superior effectiveness of Shift-Attention in DGA detection tasks.

5.7. *Performance Evaluation*

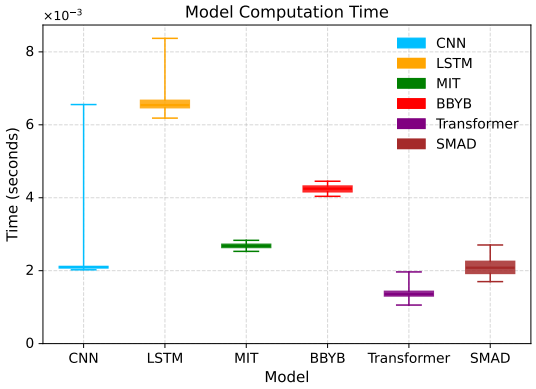


Figure 9: Model computation time of single domain name

To assess the practicality of the proposed method in real-world network environments, we implemented a real-time DGA detection prototype system and conducted system-level performance evaluations. Given the strong real-time requirements of DNS queries in practical scenarios, in addition to detection accuracy, recall, and F1-score discussed in Section 4.2, system throughput (QPS) and model inference latency also serve as key indicators for deployment feasibility. We built a stress-testing environment based on DNSperf [25] and selected 100,000 domain names of varying lengths as the input query pool to continuously evaluate the prototype system under realistic operational conditions. Specifically, these 100,000 domains constitute a pool of unique queries that may be replayed multiple times during the runtime, rather than a single-pass set of 100,000 queries. A total of 10 independent tests were performed, each running for 10,000 seconds, and the stable peak

Table 3: Comparison of Model Performance on DGA Family Classificationd

	1:1			1:3			1:4			1:7		
	ACC	Recall	F1	ACC	Recall	F1	ACC	Recall	F1	ACC	Recall	F1
CNN	0.7686	0.4400	0.3919	0.7781	0.4784	0.4227	0.7323	0.4005	0.3326	0.7686	0.4907	0.4216
LSTM	0.9130	0.7058	0.6877	0.9248	0.7442	0.7310	0.9221	0.7355	0.7236	0.9162	0.7289	0.7156
MIT	0.9255	0.7403	0.7273	0.9218	0.7357	0.7201	0.9186	0.7225	0.7080	0.9167	0.7245	0.7102
BBYB	0.8798	0.6154	0.5833	0.9014	0.6720	0.6492	0.9008	0.6718	0.6529	0.8441	0.5725	0.5390
Transformer	0.8563	0.5995	0.5780	0.8522	0.6015	0.5775	0.8407	0.5820	0.5602	0.8424	0.5980	0.5738
SMAD	**0.9348**	**0.7761**	**0.7662**	**0.9339**	**0.7746**	**0.7644**	**0.9282**	**0.7654**	**0.7528**	**0.9292**	**0.7645**	**0.7541**

QPS segments were recorded for statistical analysis. Without any DGA detection model enabled, the system sustained a baseline throughput of 3800 QPS. On top of this baseline, we enabled different detection models and measured system throughput. Based on inference latency and throughput performance, the average end-to-end QPS achieved was 609 for CNN, 147 for LSTM, 321 for MIT, 227 for BBYB, 729 for Transformer, while SMAD reached 578 QPS. These results indicate that, even under the same hardware configuration, different detection models introduce varying processing overheads, whereas SMAD significantly improves real-time processing capability due to its lightweight inference. To further evaluate the impact of inference latency on real-time deployment, we measured per-domain prediction time using Python's time library under an open-world setting, repeating the experiment 10 times and visualizing the latency distribution (as shown in Figure. 9). The results show that SMAD achieves a stable inference latency (2.3 ms), outperforming other models. BBYB and LSTM perform slower, requiring 2–3× the latency of SMAD, with LSTM showing noticeable instability. MIT, CNN, and Transformer exhibit relatively stable latency, with average inference times of 2.8 ms, 2.1 ms, and 1.3 ms, respectively. In summary, SMAD demonstrates clear advantages in both throughput and inference efficiency. By maintaining high detection performance while achieving lower computational overhead, SMAD is better suited for deployment in online DNS resolution requiring real-time DGA detection.

6. Related Work

DGA detection based on machine learning. Machine learning-based DGA detection methods can be categorized into two types: those based on traditional machine learning techniques and those utilizing deep learning approaches. The first category relies on manually extracted features of DGA domain names and traditional machine learning algorithms. For example, [26] developed a RF method for identifying malicious domain names by combining blacklist, lexical, and other features based on static domains, URLs, and semantic characteristics. R. Sivaguru and others [27] enhanced RF and LSTM feature extraction methods by utilizing side information and lexical attributes to improve robustness against adversarial attacks. KSDom [28] uses k-means to balance data and employs the CatBoost algorithm to classify domain names. Sun et al. [29] created DeepDom, which uses a Graph Convolutional Network (GCN) and Heterogeneous Information Network (HIN) model to extract complex semantics from DNS, aiding in the detection of malicious domains based on direct or indirect relationships. J. Zhu et al. [9] combined traditional SVM with self-feedback learning (F-SVM) to detect DGA domains, addressing the high cost of model updates. The second category of detection methods directly takes domain name strings as input and places them into various neural networks to automatically extract features. For instance, Highnam et al.[8] introduced 'Bilbo', a CNN and LSTM hybrid architecture used for DGA detection. In addition, Antonakakis et al. proposed Pleiades[1], which detects DGA-based malware by analyzing NXDomain traffic and combining clustering with classification, thereby enabling the discovery of new DGA families without reverse engineering.

Transformer. Since its introduction, the Transformer has been applied to classification tasks in different domains and achieved remarkable results. In the field of Web Finger Printing, some studies [30, 31] processed encrypted traffic into vectors and utilized deep learning along with the Transformer structure to transform the multi-label website fingerprinting issue into several binary classification problems, thereby achieving adaptive recognition of multi-label website fingerprints through feature extraction and modeling global interactions. In the realm of medical image classification tasks, some research works [32, 33] improved the Transformer model by introducing a hierarchical Transformer structure. By employing a masked nuclear area supervision training method, discriminative high-level nuclear features were extracted, overcoming the challenges of lacking nucleus-level labels and significant variations between histological images, resulting in satisfactory image classification outcomes. Furthermore, in music genre classification, [34] designed a Transformer classifier that effectively analyzed the relationships between different audio frames, achieving favorable results in music genre classification. These studies collectively demonstrate that the Transformer model has achieved state-of-the-art performance in sequential data classification tasks. Hence, our work attempts to utilize the Transformer model for the classification task of DGA domain names, providing a novel solution for the detection of malicious DGA domain names.

7. Conclusion

In this paper, we introduced SMAD, a robust DGA domain detection model leveraging an improved attention mechanism. SMAD adopts a Transformer architecture with a novel Shift-Attention mechanism, effectively capturing global context while mitigating adversarial obfuscation. By incorporating multi-head attention and regularization strategies, SMAD enhances generalization capability and enables accurate detection of previously unseen DGA families. Extensive experiments

on large-scale datasets demonstrate that SMAD achieves up to 96% detection accuracy and consistently superior AUC scores under adversarial settings. We further deploy SMAD in a real-time, inline DNS-layer defense system integrated with dnsdist and Bind9, validating its efficiency and practicality for operational DNS security.

Acknowledgements

This work was supported by the Beijing Nova Program under Grant 2023140, the National Natural Science Foundation of China for Youth Science Foundation Project (Class B) under Grant 62522203, the Key Program of the Beijing Natural Science Foundation (Haidian Original Innovation Joint Fund) under Grant L252013, the National Key R&D Program of China under Grant 2024YFB2906900.

References

[1] M. Antonakakis, R. Perdisci, Y. Nadji, N. Vasiloglou, S. Abu-Nimeh, W. Lee, D. Dagon, From {Throw-Away} traffic to bots: Detecting the rise of {DGA-Based} malware, in: 21st USENIX Security Symposium (USENIX Security 12), 2012, pp. 491–506.

[2] X. Jiang, S. Liu, A. Gember-Jacobson, A. N. Bhagoji, P. Schmitt, F. Bronzino, N. Feamster, Netdiffusion: Network data augmentation through protocol-constrained traffic generation, Proceedings of the ACM on Measurement and Analysis of Computing Systems 8 (1) (2024) 1–32.

[3] B. C. Cebere, J. L. B. Flueren, S. Sebastián, D. Plohmann, C. Rossow, Down to earth! guidelines for dga-based malware detection, in: Proceedings of the 27th International Symposium on Research in Attacks, Intrusions and Defenses, 2024, pp. 147–165.

[4] Y. Zhou, Q.-s. Li, Q. Miao, K. Yim, Dga-based botnet detection using dns traffic., J. Internet Serv. Inf. Secur. 3 (3/4) (2013) 116–123.

[5] C. Lever, P. Kotzias, D. Balzarotti, J. Caballero, M. Antonakakis, A lustrum of malware network communication: Evolution and insights, in: 2017 IEEE Symposium on Security and Privacy (SP), IEEE, 2017, pp. 788–804.

[6] A. Ahluwalia, I. Traore, K. Ganame, N. Agarwal, Detecting broad length algorithmically generated domains, in: Intelligent, Secure, and Dependable Systems in Distributed and Cloud Environments: First International Conference, ISDDC 2017, Vancouver, BC, Canada, October 26-28, 2017, Proceedings 1, Springer, 2017, pp. 19–34.

[7] M. Pereira, S. Coleman, B. Yu, M. DeCock, A. Nascimento, Dictionary extraction and detection of algorithmically generated domain names in passive dns traffic, in: Research in Attacks, Intrusions, and Defenses: 21st International Symposium, RAID 2018, Heraklion, Crete, Greece, September 10-12, 2018, Proceedings 21, Springer, 2018, pp. 295–314.

[8] K. Highnam, D. Puzio, S. Luo, N. R. Jennings, Real-time detection of dictionary dga network traffic using deep learning, SN Computer Science 2 (2) (2021) 110.

[9] J. Zhu, F. Zou, Detecting malicious domains using modified svm model, in: 2019 IEEE 21st International Conference on High Performance Computing and Communications; IEEE 17th International Conference on Smart City; IEEE 5th International Conference on Data Science and Systems (HPCC/SmartCity/DSS), IEEE, 2019, pp. 492–499.

[10] A. M. Saeed, D. Wang, H. A. Alnedhari, K. Mei, J. Wang, A survey of machine learning and deep learning based dga detection techniques, in: International Conference on Smart Computing and Communication, Springer, 2021, pp. 133–143.

[11] S. Vosoughi, P. Vijayaraghavan, D. Roy, Tweet2vec: Learning tweet embeddings using character-level cnn-lstm encoder-decoder, in: Proceedings of the 39th International ACM SIGIR conference on Research and Development in Information Retrieval, 2016, pp. 1041–1044.

[12] C. Xu, J. Shen, X. Du, Detection method of domain names generated by dgas based on semantic representation and deep neural network, Computers & Security 85 (2019) 77–88.

[13] J. Woodbridge, H. S. Anderson, A. Ahuja, D. Grant, Predicting domain generation algorithms with long short-term memory networks, arXiv preprint arXiv:1611.00791 (2016).

[14] B. Yu, D. L. Gray, J. Pan, M. De Cock, A. C. Nascimento, Inline dga detection with deep networks, in: 2017 IEEE international conference on data mining workshops (ICDMW), IEEE, 2017, pp. 683–692.

[15] H. Shahzad, A. R. Sattar, J. Skandaraniyam, Dga domain detection using deep learning, in: 2021 IEEE 5th International Conference on Cryptography, Security and Privacy (CSP), IEEE, 2021, pp. 139–143.

[16] D. S. Berman, Dga capsnet: 1d application of capsule networks to dga detection, Information 10 (5) (2019) 157.

[17] X. Yun, J. Huang, Y. Wang, T. Zang, Y. Zhou, Y. Zhang, Khaos: An adversarial neural network dga with high anti-detection ability, IEEE transactions on information forensics and security 15 (2019) 2225–2240.

[18] L. Nie, X. Shan, L. Zhao, K. Li, Pkdga: A partial knowledge-based domain generation algorithm for botnets, IEEE Transactions on Information Forensics and Security (2023).

[19] Y. Zhai, J. Yang, Z. Wang, L. He, L. Yang, Z. Li, Cdga: A gan-based controllable domain generation algorithm, in: 2022 IEEE International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom), IEEE, 2022, pp. 352–360.

[20] A. O. Almashhadani, M. Kaiiali, D. Carlin, S. Sezer, Maldomdetector: A system for detecting algorithmically generated domain names with machine learning, Computers & Security 93 (2020) 101787.

[21] A. AlSabeh, K. Friday, E. Kfoury, J. Crichigno, E. Bou-Harb, On dga detection and classification using p4 programmable switches, Computers & Security 145 (2024) 104007.

[22] P. Mockapetris, Rfc1035: Domain names-implementation and specification (1987).

[23] DGArchive, Dgarchive dataset, Web, accessed: 2023-04-28 (2023). URL `https://dgarchive.caad.fkie.fraunhofer.de/`

[24] H. S. Anderson, J. Woodbridge, B. Filar, Deepdga: Adversarially-tuned domain generation and detection, in: Proceedings of the 2016 ACM workshop on artificial intelligence and security, 2016, pp. 13–21.

[25] DNSPerf Contributors, Dnsperf/dnsperf: Dns performance testing tool, `https://github.com/DNSPerf/dnsperf`, gitHub repository.

[26] N. F. Ghalati, N. F. Ghalaty, J. Barata, Towards the detection of malicious url and domain names using machine learning, in: Technological Innovation for Life Improvement: 11th IFIP WG 5.5/SOCOLNET Advanced Doctoral Conference on Computing, Electrical and Industrial Systems, DoCEIS 2020, Costa de Caparica, Portugal, July 1–3, 2020, Proceedings 11, Springer, 2020, pp. 109–117.

[27] R. Sivaguru, J. Peck, F. Olumofin, A. Nascimento, M. De Cock, Inline detection of dga domains using side information, IEEE Access 8 (2020) 141910–141922.

[28] Q. Wang, L. Li, B. Jiang, Z. Lu, J. Liu, S. Jian, Malicious domain detection based on k-means and smote, in: Computational Science–ICCS 2020: 20th International Conference, Amsterdam, The Netherlands, June 3–5, 2020, Proceedings, Part II 20, Springer, 2020, pp. 468–481.

[29] X. Sun, Z. Wang, J. Yang, X. Liu, Deepdom: Malicious domain detection with scalable and heterogeneous graph convolutional networks, Computers & Security 99 (2020) 102057.

[30] Z. Jin, T. Lu, S. Luo, J. Shang, Transformer-based model for multi-tab website fingerprinting attack, in: Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security, 2023, pp. 1050–1064.

[31] X. Deng, Q. Yin, Z. Liu, X. Zhao, Q. Li, M. Xu, K. Xu, J. Wu, Robust multi-tab website fingerprinting attacks in the wild, in: 2023 IEEE Symposium on Security and Privacy (SP), IEEE, 2023, pp. 1005–1022.

[32] Y. Bai, Y. Mi, Y. Su, B. Zhang, Z. Zhang, J. Wu, H. Huang, Y. Xiong, X. Gong, W. Wang, A scalable graph-based framework for multi-organ histology image classification, IEEE Journal of Biomedical and Health Informatics 26 (11) (2022) 5506–5517.

[33] Y. Su, Y. Bai, B. Zhang, Z. Zhang, W. Wang, Hat-net: A hierarchical transformer graph neural network for grading of colorectal cancer histology images., in: BMVC, 2021, p. 412.

[34] Y. Zhuang, Y. Chen, J. Zheng, Music genre classification with transformer classifier, in: Proceedings of the 2020 4th international conference on digital signal processing, 2020, pp. 155–159.